

生成解码与KV缓存实验手册

第056单元 从概率到缓存的独立复现

本实验固定一个20词元、2层、宽度16、4头、7输入位置的小型decoder，比较解码和缓存实现。权重来自本课程055公开检查点，已放入data/weights.npz；本实验不用训练、不访问网络、不收费。你将提交手算、160条生成结果、缓存误差和固定工作量计时，并解释反例。

先修：理解下一个token预测、softmax、数组shape。正文中的“缓存”指每层保存的key/value张量；“EOS”是停止符，ID为2。正式prompt为[BOS,颜色1]，最多新增6token，新增数包含EOS。

一 环境与第一轮运行

解压单元包后，终端切换到包含experiment.py的056目录。不要从别的目录猜相对路径。推荐建立独立环境；requirements.txt固定实测核心版本，environment.yml提供Conda入口。CPU即可，建议预留1GB内存和100MB课程文件空间；PyTorch安装体积另计。

```
conda env create -f environment.yml
conda activate dl056
python test_experiment.py
python -O test_experiment.py
python experiment.py --output replay
python make_figures.py --results replay/results.json --output replay/figures
jupyter lab experiment.ipynb
```

若用已有Python3.12环境，可运行python -m pip install -r requirements.txt。PyTorch CPU专用轮子可按官方安装器选择；本次实测torch2.14.1+cpu，requirements中的2.14.1允许相应本地版本标记。没有GPU不影响本课。Notebook清空后Run All会在notebook_replay重跑，保留随包outputs原结果。

图表要求Noto Sans CJK字体，默认路径为/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc。在其他操作系统，安装官方Noto字体后，把DL_CJK_FONT设为该字体的绝对路径。PDF已随包提供；重建PDF才需要build-tools/requirements.txt及CJK Serif字体，不属于运行实验的必要步骤。

预期结果是测试打印PASS、replay/results.json存在且有160条samples、两种实现输出相同。执行通常数秒到一分钟，随硬件不同。计时不是固定数值验收项。若测试不通过，先保留错误信息，不通过删断言获得“成功”。

二 先在纸上建立四词元解码

已知 $p=[0.4, 0.3, 0.2, 0.1]$ 。完成以下六问，每问都写中间量：

1. 选择 $\text{logit}=\log(p)$ 时，为什么softmax恰好还原 p ？
2. $T=0.5$ 时四项未归一化权重是多少？它们的和是多少？
3. top-k2保留哪两项？过滤后概率是多少？
4. top-p0.6何时停止累加？为什么跨阈值的B必须保留？
5. $T=0.5$ 以后再做top-p0.6，结果是否仍为4/7和3/7？
6. 原分布的累计区间中，均匀随机数0.65落在哪个词元？

先独立完成，再调用`experiment.distribution`核对。输出是float64，允许小于 $1e-12$ 的舍入差。 $T=0$ 是非法采样参数；贪心用单独配置`{'greedy': True}`。

三 跟踪一次真正的生成

打开`outputs/protocol.json`，抄下第一个prompt、禁止输出的ID、种子范围、权重SHA和`max_new`。查看`outputs/results.json`中该prompt的贪心和T1.3样本，把token ID按`data/corpus.json`的vocab翻译成符号。

按步骤画出输入长度2、3、4、5、6、7对应的六次预测。每次说明取哪个logit、何时追加token、何时判断EOS。若第六次才生成EOS，最终序列长度是多少？为什么位置表只有7仍然合法？如果第三次就生成EOS，为什么不再运行第四次？

然后重跑同一配置、同一种子，比较tokens列表；不要要求seconds字段相同。换种子后也可能偶然得到同一输出，不能用“必须不同”作为随机采样正确性的测试。

四 缓存和mask手工对齐

取 $B=2$ 、 $H=4$ 、 $d_h=4$ ，已有 $P=3$ 个位置，新chunk长 $C=2$ 。

1. 写出新Q、拼接后K/V、attention分数的shape。
2. 画出 2×5 的allowed矩阵，1表示可见，0表示屏蔽。
3. 写出新chunk的位置索引。
4. 说明为什么每一层要有自己的缓存，为什么不直接复用上一层K/V。
5. 运行`test_experiment.py`中的B2分块核验；把完整前向与拼接分块输出按元素比较。

常见错误是把新chunk的索引重置为0、忘记把旧K/V放在拼接前方、套错方形因果mask、在训练模式运行缓存。测试还修改最后token，检查前面所有logit不变，以捕捉未来泄漏。

五 算出纯缓存载荷

使用 $M=2 \times \text{层数} \times \text{批大小} \times \text{长度} \times \text{宽度} \times \text{每元素字节数}$ 。

- 对本模型 $B=1$ 、 $T=7$ 、float32，手算字节数和KiB
- 将 B 改成2，再核对test_experiment.py中的实际 $\text{tensor.numel} \times \text{element_size}$
- 若改为float16，理论载荷减半；解释为什么这不等于实际端到端推理内存减半
- 列举纯载荷遗漏的至少三项内存

图03的 T 到128是理论曲线。不要直接给本检查点输入128位置来“验证”，因为它只学过长7的位置表。

六 公平比较延迟和退化

正式比较五组解码，每组 $4 \times \text{prompt} \times 8 \text{ seed}$ ，共32条。对每组计算：平均新增token数、EOS终止比例、prompt与续写不同组合数、相邻重复次数、重复二元组比例。保存所有样本，不能删除不好看的输出。

解释 $T1.3$ 为何既有未正常结束的输出，又可能有更短的平均长度。构造一个相邻重复为1而重复二元组比例为0的短序列，说明为什么两项要并报。

计时另走固定7步工作量。每个实现3次warm-up，随后20次交替测量；包含Python循环和缓存拼接，不包括模型加载、写文件和网络。画出原始计时分布，报告中位数。若缓存与无缓存非常接近、或缓存更慢，保留结果并解释短序列开销，不追加只对某实现有利的重复次数。

七 验收与常见排错

最低提交物：六问手算； $2 \times 5 \text{ mask}$ 和shape；完整replay/results.json；五幅从该JSON重画的图；一页结果解释；清空后顺序执行的Notebook。结论必须同时覆盖概率过滤、缓存等价、停止边界和负面结果。

现象	优先检查	不应采取的做法
$T=0$ 报错	用显式greedy分支	给除零加极小数冒充贪心
cache shape mismatch	批大小、层数、缓存长度是否一致	随意reshape“修复”
position capacity exceeded	$\text{prompt} + \text{max_new} - 1$ 是否大于7	无说明地截掉历史
全部概率NaN	是否把所有logit屏蔽	任意改成均匀分布掩盖错误
缓存样本不同	先比logit、位置、mask、RNG消耗	只换seed让文本碰巧一致
图中文变方框	CJK字体与DL_CJK_FONT	将错误图当有效证据

进一步思考但不要求扩大实验：真实服务中批内结束时间不同、beam重排缓存、滑动窗口、KV量化会增加哪些契约？写清可能影响缓存内容、位置或数值精度的因素即可，不需要购买算力或调用大模型。

八 两维注意力与缓存身份检查

用讲义第九节的手造Q/K/V，独立算出三个内积、缩放分数、softmax和两维输出。用torch.float64重算并核对；把结果四舍五入到六位小数。说明这里的value加权和为何还不是词表概率。

再做一个不修改正式结果的隔离实验：给两个不同、等长prompt分别建立缓存，然后故意把A的缓存用于B的后续token。记录这种错误可能不触发shape异常，却不再等价于B完整前向的原因。只在单独文件中做此错误注入，不更改正式experiment.py，也不要将错误结果混入原对照。

最后写出至少四种缓存应失效的情况，并说明eval与no_grad各自负责什么。提交时标明这是机制练习，不是新增模型性能测量。